

Neural Graph Generation with conditioning

Ibrahim RIDENE

Omar ARBI

Malek BOUHADIDA

January 15, 2025

1 INTRODUCTION

Graph generation with specific properties is a critical task with applications in fields like bioinformatics, social network analysis, and molecular design. This project focuses on generating graphs that align with natural language descriptions of properties such as the number of nodes, edges, clustering coefficients, and other structural metrics, bridging the gap between textual inputs and graph data to facilitate intuitive graph creation. The task’s primary evaluation metric, Mean Absolute Error (MAE), measures the alignment between generated graphs and their ground-truth property vectors, posing the challenge of effectively combining textual data with graph representations. To address this, we propose an architecture based on a Variational Graph AutoEncoder (VGAE) augmented with a latent diffusion model, where the VGAE extracts compressed graph representations and the diffusion model iteratively refines them using conditioning from textual descriptions. This approach enables the generation of property-compliant and structurally valid graphs. The report details the dataset, methodology, training process, results, and potential future improvements, highlighting the broader implications of this work in structured data generation.

2 DATA

Each sample in the dataset comprises an undirected graph represented as an edge list and a textual description detailing seven key properties: Number of nodes, Number of edges, Average degree, Number of triangles, Global clustering coefficient, Maximum k -core, Number of communities. The dataset is split into:

- **Training set:** 8000 samples with graphs and descriptions.
- **Validation set:** 1000 samples used for hyperparameter tuning and preventing overfitting.
- **Test set:** 1000 samples containing only descriptions, used to evaluate graph generation.

Graphs in the dataset are medium-sized. Table 1 summarizes key statistics of the graph properties.

Table 1: Descriptive statistics of graph properties in the training set.

Property	Max	Min	Mean	Std Dev
Nodes	50	10	30.6	11.8
Edges	1225	5	226.2	234.4
Average Degree	49.0	1.0	12.9	10.2
Triangles	19600	0	1389.9	2795.0
Global Clustering Coefficient	1.0	0.0	0.5	0.3
Maximum k -core	49.0	1.0	11.4	9.9
Communities	16.0	1.0	3.3	1.4

3 METHOD

3.1 Overview

In this section, we detail the methodologies employed in our approach. The baseline model, rooted in the Neural Graph Generator (NGG) framework [1], integrates two principal components:

- **Variational Graph AutoEncoder (VGAE):** This module generates a compressed latent representation for each graph, effectively capturing its structural features.
- **Latent Diffusion Model:** Operating within the latent space, this component refines the embeddings through a diffusion process conditioned on the properties vector. The final graph is subsequently reconstructed by the VGAE decoder using the refined latent representation.

This architecture facilitates the generation of graphs that adhere closely to the specified properties by leveraging the complementary strengths of the VGAE and diffusion mechanisms.

3.2 Graph Description Embedding

The conditioning of the input description is pivotal for generating graphs that align with the specified properties. Rather than extracting the seven graph properties and encoding them into a fixed 7-dimensional vector, we encode the entire textual description into a 768-dimensional embedding using **bert-base-uncased**.

This pre-trained language model, developed using masked language modeling (MLM) on extensive English text corpora, captures nuanced semantic information from the textual descriptions. By doing so, it provides a more comprehensive representation of the graph’s underlying properties. The resulting embeddings are subsequently utilized as conditioning codes within the NGG framework to ensure accurate and property-compliant graph generation.

3.3 GAT based encoder

The VGAE in the baseline model used Graph Isomorphism Networks (GIN) in the encoder. GIN operates by aggregating node features through a sum-based aggregation function, combined with a learnable transformation. While it is effective to distinguish graph structures, its uniform weighting of neighbours limits its flexibility specifically in tasks where certain neighbours play a more critical role than others.

Graph Attention Networks (GAT) address this limitation by introducing an attention mechanism that dynamically weights neighbors, enabling the model to focus on the most relevant connections. Let G be an undirected graph with N nodes. Let d be the dimension of the node embeddings, and $h_1, \dots, h_N \in \mathbb{R}^d$ be the embeddings themselves. Let $W \in \mathbb{R}^{d \times d}$ and $a \in \mathbb{R}^{2d}$. The attention weights are:

$$e(h_i, h_j) = a^\top \text{LeakyReLU}([Wh_i \| Wh_j]) \quad (1)$$

where $\|$ represents the concatenation operator. The attention weights are then normalized using softmax:

$$\alpha_{ij} = \frac{\exp(e(h_i, h_j))}{\sum_{k \in \mathcal{N}_i} \exp(e(h_i, h_k))} \quad (2)$$

where $\mathcal{N}_i$ is the set of neighbours of node i in G . The embedding of node i is then calculated as follows:

$$h'_i = \text{LeakyReLU} \left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} Wh_j \right) \quad (3)$$

In reality, we generally use multi-head attention, with K heads, $a^{(1)}, \dots, a^{(K)} \in \mathbb{R}^{2d/K}$ and $W^{(1)}, \dots, W^{(K)} \in \mathbb{R}^{d \times d/K}$:

$$h'_i = \text{LeakyReLU} \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ij}^{(k)} W^{(k)} h_j \right) \quad (4)$$

And to obtain better results, we will stack multiple GAT layers to obtain a deeper model.

3.4 Conditioned Decoder

The default VAE decoder lacks conditioning based on the embedded prompt description of the graph. As a result, we made some modifications by adding conditioning to the decoder that incorporates contextual constraints encapsulated within the conditioning vector, enhancing the generation process by aligning graphs with specified properties. This approach integrates meaningful external features into the latent space representation, ensuring that graph generation is guided by both latent variables and explicit contextual information. This approaches helped to get better performance.

3.5 Improved Denoising Conditioning

As our task consists of a conditional generation of graphs, we not only want to generate graphs, but we want them to respect the specified properties. The considered latent diffusion model already does the conditioning using concatenation. Another approach we will experiment is called Feature-wise Linear Modulation (FiLM) which is a conditioning mechanism that modulates the features of the model by applying scale and shift transformations, based on the conditioning vector. Given a feature representation h and a conditioning vector c , FiLM computes the modulated features as:

$$h' = \gamma(c) \odot h + \beta(c)$$

where $\gamma(c)$ (resp. $\beta(c)$) is a learned function that generates scaling factors (resp. shift factors) based on the conditioning vector and $\odot$ is the element-wise multiplication.

In fact, this procedure allows the model to perform the conditioning adaptively. Unlike simple concatenation, which merely augments the representation, FiLM actively transforms the latent features to incorporate the desired properties.

And to go even further we will also include the conditioning in the VGAE, this time feeding the concatenation of the conditional vector and the latent graph representation to the decoder [2]

3.6 Attention mechanism and scheduler refinement for the Denoiser

The denoiser in the baseline model already demonstrated strong performance, so we will just opt for subtle modifications to further improve it. We will use multi-head attention within the neural network, allowing the model to attend to different parts of the input simultaneously, thereby improving its ability to model graph patterns during the denoising process.

Additionally, we will switch the noise scheduling from a linear scheduler to a cosine scheduler, both defined as follows, We define the noise schedulers as follows, where $\beta_{\min}$ and $\beta_{\max}$ are the minimum and maximum noise levels and s is a small stability constant

- **Linear Scheduler:**

$$\beta_t = \beta_{\min} + t \cdot \frac{\beta_{\max} - \beta_{\min}}{T}$$

- **Cosine Scheduler:**

$$\beta_t = \beta_{\min} + \left(\frac{\cos\left(\frac{\pi t}{T} + s\right)}{\cos(s)} \right)^2 \cdot (\beta_{\max} - \beta_{\min})$$

The cosine scheduler, inspired by advancements in diffusion models, modifies the noise over time following a cosine curve which results in a more gradual and smoother reduction in noise compared to the linear scheduler, allowing the model to focus more effectively on fine details during the later stages of denoising.

3.7 Language Model Fine-tuning

As mentioned in a previous subsection, we used a pre-trained language model to encode the graph description instead of passing the properties in a single vector. A natural track to follow is to fine-tune the language model, namely **bert-base-uncased** on our specific dataset. The idea is that since the starting pretrained language model was trained on a large corpus of text, the semantic information concerning graph structures was most likely completely undifferentiated, therefore lending similar embeddings to different graphs. Fine-tuning the model on our dataset will allow learning a more specialized representation of the text, and therefore yield better results.

4 TRAINING

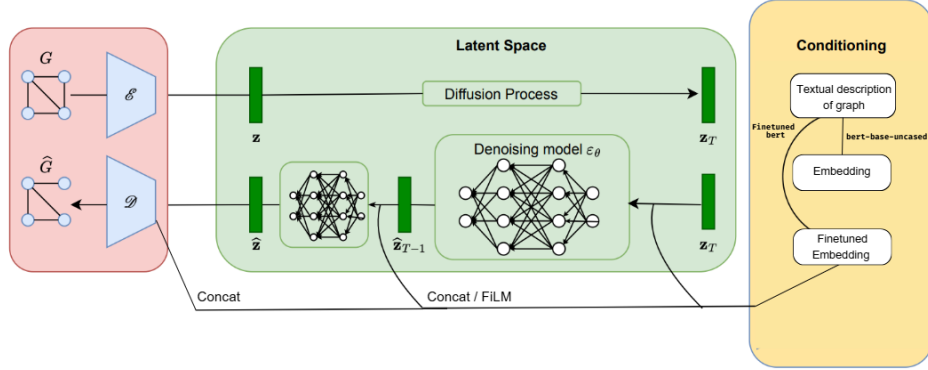


Figure 1: Model architecture, initially based on GNN and further improved [1].

The training process fine-tunes the Neural Graph Generator (NGG) to generate graphs that align with textual descriptions. Textual descriptions are encoded into 768-dimensional embeddings using the pre-trained `bert-base-uncased` model, while input graphs are processed into latent representations via a Graph Attention Network (GAT) encoder. The latent diffusion model iteratively refines these representations, incorporating adaptive conditioning through Feature-wise Linear Modulation (FiLM) to enforce property-specific constraints.

The model is trained to minimize the Mean Absolute Error (MAE) between generated and target graph properties, with additional regularization to maintain smooth latent space representations and prevent overfitting. The training involves the following steps:

- **Forward Pass:** Textual descriptions and graphs are encoded and processed through the diffusion model and decoder to generate graphs.
- **Backward Pass:** Gradients are computed using the loss function, and the parameters of the GAT encoder, diffusion model, and decoder are updated through backpropagation.
- **Validation:** The validation set evaluates the generalization of the model and informs hyperparameter tuning to optimize performance.

On the test set, where graphs are generated solely from textual descriptions, the model achieves an MAE of **0.17992**, demonstrating its effectiveness in aligning generated graphs with specified properties. By leveraging this iterative training process, the NGG framework ensures robust and property-compliant graph generation.

5 RESULTS

In this section, we will perform the different training and validation steps for the different models implemented. We have especially 3 different types of VAE and 5 variants of denoisers. For more details about the models see Appendix [8].

Figure 2 shows the different validation Losses when training the different types of Autoencoders. The validation loss curves show that the GIN Encoder (green) achieves the lowest and most stable validation loss, indicating its superior performance in capturing graph structures. The GIN Encoder’s sum-based aggregation mechanism effectively models graph isomorphism, leading to better generalization. The GAT Encoder (blue) performs slightly worse, showing a marginally higher validation loss with small fluctuations, likely due to its attention mechanism, which, while powerful, can be less efficient in some graph structures. The Default Autoencoder (red) underperforms significantly, stabilizing at a much higher validation loss, demonstrating its limitations in leveraging graph-specific features. This is mainly due to the fact that this Autoencoder is not conditioned on the embedding of the prompt.

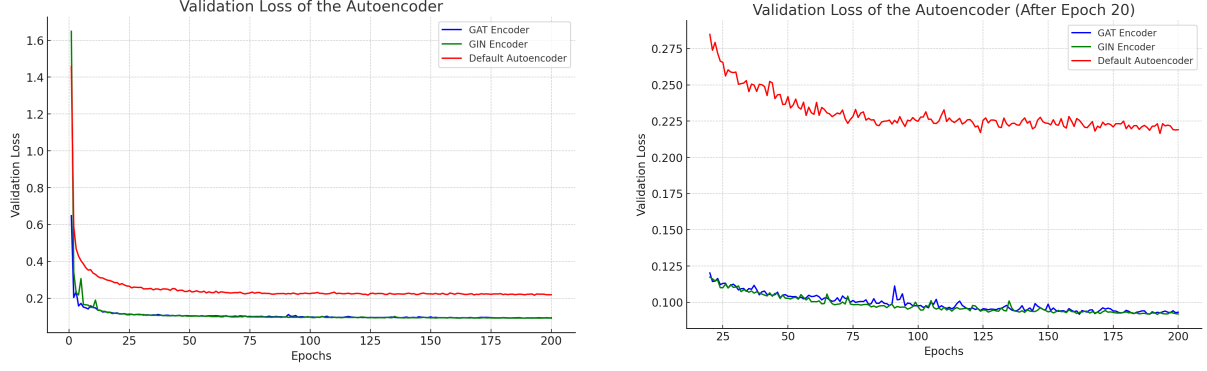


Figure 2: Validation performance across 200 epochs for each AE architecture.

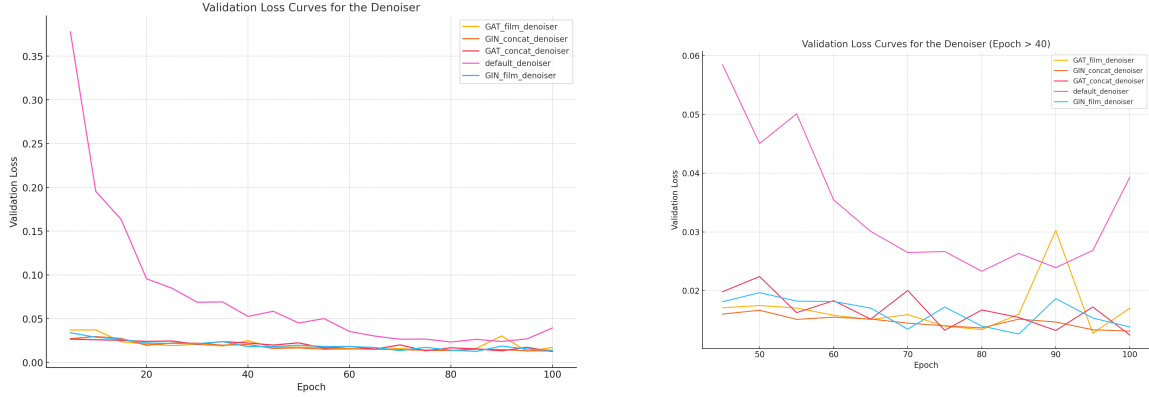


Figure 3: Validation performance across 100 epochs for each Denoiser architecture.

Figure 3 represents the evolution of the validation loss for the different architecture of the Denoiser. All models (except the default) have good performance due to the fact that they take a good embedding representation of the prompt based on the finetuned BERT model on this specific task. The validation loss curves indicate that the GIN Concat denoiser (orange) provides better performance and greater stability compared to other configurations. This suggests that concatenating the conditional input with the activations allows the model to explicitly combine graph features and prompt embeddings without introducing additional complexity or variability. The GIN encoder’s inherently expressive sum-based aggregation complements this approach, enabling the network to effectively capture graph structural information while maintaining stable integration of conditional inputs. This stability makes concatenation a reliable and robust choice for denoising tasks. The spike at the end for the GAT FiLM denoiser (yellow) can be attributed to the combination of GAT’s attention mechanism and FiLM’s dynamic modulation, which lead to instability and overfitting problem due to the complexity of the model. The Default Denoiser (pink) underperforms significantly, stabilizing at a much higher validation loss, demonstrating its limitations in leveraging graph-specific features. This is mainly due to the fact that this Denoiser is conditioned on the simple embedding of the prompt represented by the vector of 7 features which do not enable better representation of the information given in the input prompt.

6 FUTURE IMPROVEMENTS

Future directions for this work include using larger pre-trained language models, such as **bert-large** or **Mixtral-8x7B-Instruct**, to capture richer semantic nuances from textual descriptions. Enhancing scalability to support larger and more complex graphs while optimizing computational efficiency through techniques like pruning or quantization is also critical.

Additionally, incorporating uncertainty quantification could improve interpretability, and exploring advanced diffusion mechanisms or alternative evaluation metrics, such as Graph Edit Distance, could further refine model performance.

Finally, extending the framework to handle diverse graph types and applying it to real-world domains

would validate its adaptability and broaden its practical impact.

7 CONCLUSION

In conclusion, the following key takeaways summarize the outcomes of this challenge:

- The **GAT-layered encoder** demonstrated superior performance compared to the GIN-layered encoder, effectively capturing both local and global graph structures.
- The importance of **conditioning** in graph generation problems was evident, as significant improvements were achieved by employing a Conditioned VGAE over a simple VGAE.
- **Computational resources** remain a significant bottleneck. While the best results were obtained with the `bert-base-uncased` model, we believe that the performance of the pipeline could be significantly enhanced with greater computational resources, allowing for the use of larger models such as `bert-large` or `Mixtral-8x7B-Instruct-v0.1`.

References

- [1] Iakovos Evdaimon et al. “Neural Graph Generator: Feature-Conditioned Graph Generation using Latent Diffusion Models”. In: (2024). URL: <https://doi.org/10.48550/arXiv.2403.01535>.
- [2] Martin Simonovsky and Nikos Komodakis. “GraphVAE: Towards Generation of Small Graphs Using Variational Autoencoders”. In: (2018).

8 Appendix

8.1 Training performances

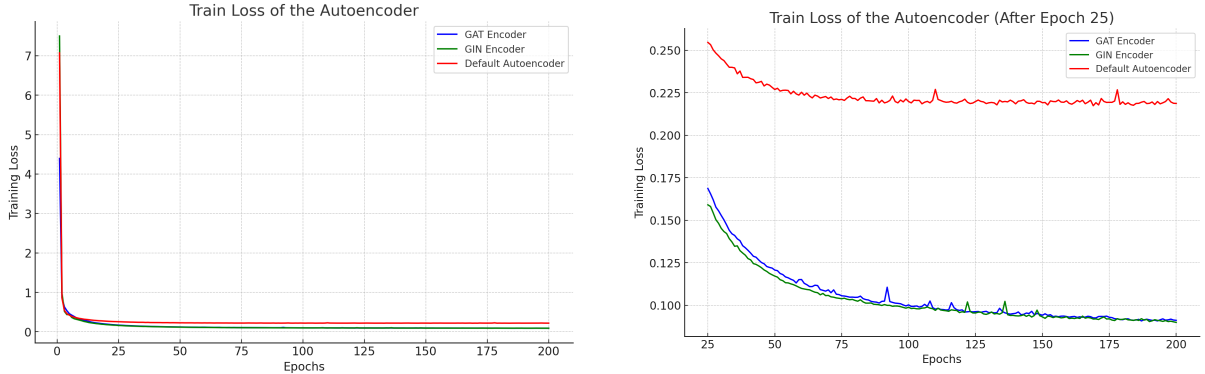


Figure 4: Train performance across 100 epochs for each denoiser architecture

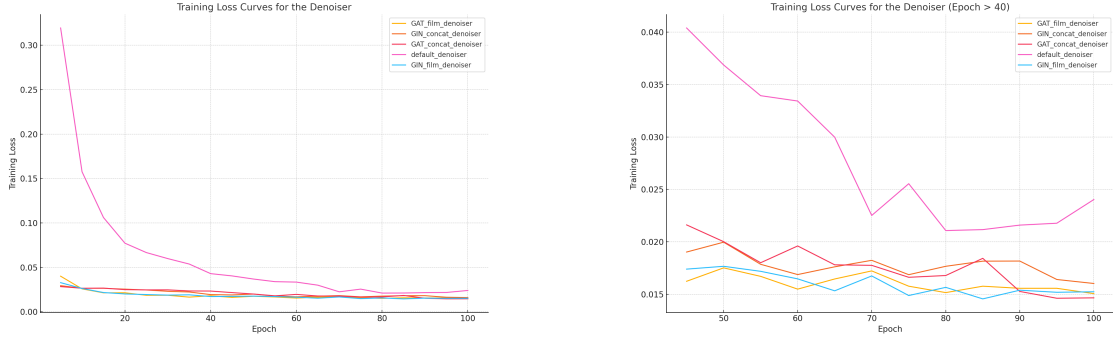


Figure 5: Train performance across 100 epochs for each denoiser architecture

8.2 Model details

We used 3 different architectures of VGAE:

- **Default Autoencoder:** This model represent the default architecture where we have GIN layers in the Encoder and a non-conditioning decoder. The embedding of the prompt here takes its default representation with a vector of 7 values describing the graph.
- **GIN Autoencoder:** The improved architecture with GIN Encoder, a conditioning decoder that takes into account the embedding of the prompt describing the graph architecture. This model is linked to a pre-trained BERT model to do the prompt embedding. Note that this embed model is Finetuned during the training process of the Autoencoder.
- **GAT Autoencoder:** The improved architecture with GAT Encoder, a conditioning decoder that takes into account the embedding of the prompt describing the graph architecture. This model is linked to a pre-trained BERT model to do the prompt embedding. Note that this embed model is Finetuned during the training process of the Autoencoder.

as well as 5 different types of Denoiser:

- **Default Denoiser:** This model represent the default architecture of denoiser where we concat the simple embedding of the prompt with the activations. The embedding of the prompt here takes its default representation with a vector of 7 values describing the graph.

- **GIN concatenation Denoiser:** This is an improvement implementation of the denoiser by adding a multi-head attention block, as well as taking a good embedding representation of the prompt based on the finetuned BERT model (finetuned when training the Autoencoder with GIN encoder). Here, we consider a concatenation method between the activations and the condition vector.
- **GAT concatenation Denoiser:** This is an improvement implementation of the denoiser by adding a multi-head attention block, as well as taking a good embedding representation of the prompt based on the finetuned BERT model (finetuned when training the Autoencoder with GAT encoder). Here, we consider a concatenation method between the activations and the condition vector.
- **GIN FiLM Denoiser:** This is an improvement implementation of the denoiser by adding a multi-head attention block, as well as taking a good embedding representation of the prompt based on the finetuned BERT model (finetuned when training the Autoencoder with GIN encoder). Here, we consider a Feature-wise Linear Modulation (FiLM) method between the activations and the condition vector.
- **GAT concatenation Denoiser:** This is an improvement implementation of the denoiser by adding a multi-head attention block, as well as taking a good embedding representation of the prompt based on the finetuned BERT model (finetuned when training the Autoencoder with GAT encoder). Here, we consider a Feature-wise Linear Modulation (FiLM) method between the activations and the condition vector.